Digital Alarm Clock

Objective

The objective of this project is to design and implement a Digital Alarm Clock using verilog HDL and to verify its functionality through simulation, synthesis and physical design. The system accurately counts seconds, minutes and hours in a 24 hour format, allows user defined time initialization, and generates an alarm signal at a preset time for a fixed duration.

The project also aims to apply the complete VLSI design flow, including RTL modeling, test bench verification, timing analysis, RTL synthesis, power analysis and physical layout, ensuring that the design is functionally correct. synthesizable and physically realizable.

Introduction

A digital alarm clock is a synchronous digital system designed to keep track of time and generate an alert at a user defined moment. In digital system design, time is represented using binary counters driven by a clock signal, making such systems a practical application of sequential logic circuits. The project focuses on implementing a digital alarm clock using Verilog Hardware Description Language (HDL) and realizing it through a standard VLSI design flow.

In this design, a clock signal provides periodic timing reference, while a reset signal initializes the system to a known state. The time is maintained using registers that store seconds, minutes and hours operating in a 24 hour format. When the seconds counter reaches its maximum value, it resets and increments the minutes counter, and similarly minutes increment the hour counter. This cascading behavior is a fundamental concept in digital counters.

An alarm mechanism is incorporated by comparing the current time with a preset alarm time. When both hour and minutes values match the alarm settings an alarm output signal is asserted for a predefined duration. This comparison logic demonstrates the use of combinational logic within a sequential system. Registers, counters, comparators and control logic together form the core building blocks of this design.

Terminologies:

- **Clock Signal**: A periodic digital signal that synchronizes all operations in the digital system.
- **Reset Signal:** A control signal used to initialize the system to a known state.
- **Sequential Logic:** Digital logic in which outputs depend on both current input and previous states.
- **Register:** A storage element used to hold binary data such as time value.
- **Counter:** A sequential circuit that increments its value on each clock cycle.

- **Alarm time:** A user defined hour and minute at which the alarm is triggered.
- **Verilog HDL:** A hardware description language used to model, simulate and synthesize digital circuits.
- **Testbench:** A verilog module used to verify the functional correctness of the design through simulation.
- **RTL (Register Transfer Level)**: A level abstraction describing data transfers between registers.
- **RTL Synthesis:** The process of converting RTL code into a gate level netlist.
- **Physical Design:** The stage where the synthesized design is converted into a chip through floor planning, placement and routing.
- **Floor planning:** The process of defining the chip area and placing major design blocks to optimize performance, power and area.
- **Power ring:** A closed loop power distribution structure placed around the core to supply stable VDD and GND.
- **Power stripes:** Vertical or horizontal metal lines used to distribute power evenly across the chip core.
- **Routing:** The process of creating metal interconnections between placed standard cells based on the netlist.

Software Requirements

1. Cadence
2. Xming
3. Putty

Working Procedure

The project was initiated by developing the Verilog code of the alarm clock based on the fundamental concepts learned in laboratory sessions and theory classes. After implementing the basic structural modules such as clock generation and time counters, additional fine tuning was performed to customize the design according to our project requirements and preferences. During functional verification, a testbench was created and timing waveforms were observed in Cadence. At this stage, an issue was encountered where the time intervals in the waveform were not uniform, making it difficult to verify the one-second clock cycle. After studying the simulation time scale settings, the time scale was adjusted to milliseconds, which resulted in a clearly defined one-second clock period. This correction allowed accurate observation of time progression and proper verification of the alarm triggering condition.

Once the design was functionally validated, RTL synthesis was carried out in Cadence to generate the gate-level representation of the alarm clock according to our lab manual. Finally, the synthesized design was used to create the physical layout in Cadence, completing the full design flow from behavioral description to physical implementation.

Verilog Code & Testbench

Verilog Code:

```
`timescale 1s / 1ms
module digital_clock (
    input clk,
    input reset,
    input [5:0] reset_min,
    input [4:0] reset_hour,
    input [5:0] alarm_min,
    input [4:0] alarm_hour,
    output reg [5:0] sec,
    output reg [5:0] min,
    output reg [4:0] hour,
    output reg alarm
);

    reg [5:0] alarm_counter;

    always @(posedge clk) begin
        if (reset) begin
            sec  <= 0;
            min  <= reset_min;
            hour <= reset_hour;
            alarm <= 0;
            alarm_counter <= 0;
        end
        else begin
            // Time counting
            if (sec == 59) begin
                sec <= 0;
                if (min == 59) begin
                    min <= 0;
                    if (hour == 23)
                        hour <= 0;
                    else
```

```
                        hour <= hour + 1;
                end
                else
                    min <= min + 1;
            end
            else
                sec <= sec + 1;

            // Alarm logic
            if (hour == alarm_hour && min == alarm_min &&
alarm_counter == 0) begin
                alarm <= 1;
                alarm_counter <= 1;
            end
            else if (alarm_counter > 0) begin
                alarm_counter <= alarm_counter + 1;
                if (alarm_counter == 60) begin
                    alarm <= 0;
                    alarm_counter <= 0;
                end
            end
        end
    end

endmodule
```

Testbench:

```
`timescale 1s / 1ms
module digital_clock_tb;

reg clk;
reg reset;
reg [5:0] reset_min;
reg [4:0] reset_hour;
reg [5:0] alarm_min;
reg [4:0] alarm_hour;

wire [5:0] sec;
wire [5:0] min;
wire [4:0] hour;
wire alarm;
```

```
digital_clock dut (
    .clk(clk),
    .reset(reset),
    .reset_min(reset_min),
    .reset_hour(reset_hour),
    .alarm_min(alarm_min),
    .alarm_hour(alarm_hour),
    .sec(sec),
    .min(min),
    .hour(hour),
    .alarm(alarm)
);

initial begin
    clk = 0;
    forever #0.5 clk = ~clk;
end

initial begin
    $shm_open("shm.db",1);
    $shm_probe("AS");
    #120 $finish;
    $shm_close();
end

initial begin
    reset = 1;
    reset_hour = 5'd10;
    reset_min  = 6'd58;
    alarm_hour = 5'd10;
    alarm_min  = 6'd59;
    #2 reset = 0;
end

endmodule
```

Timing Diagram

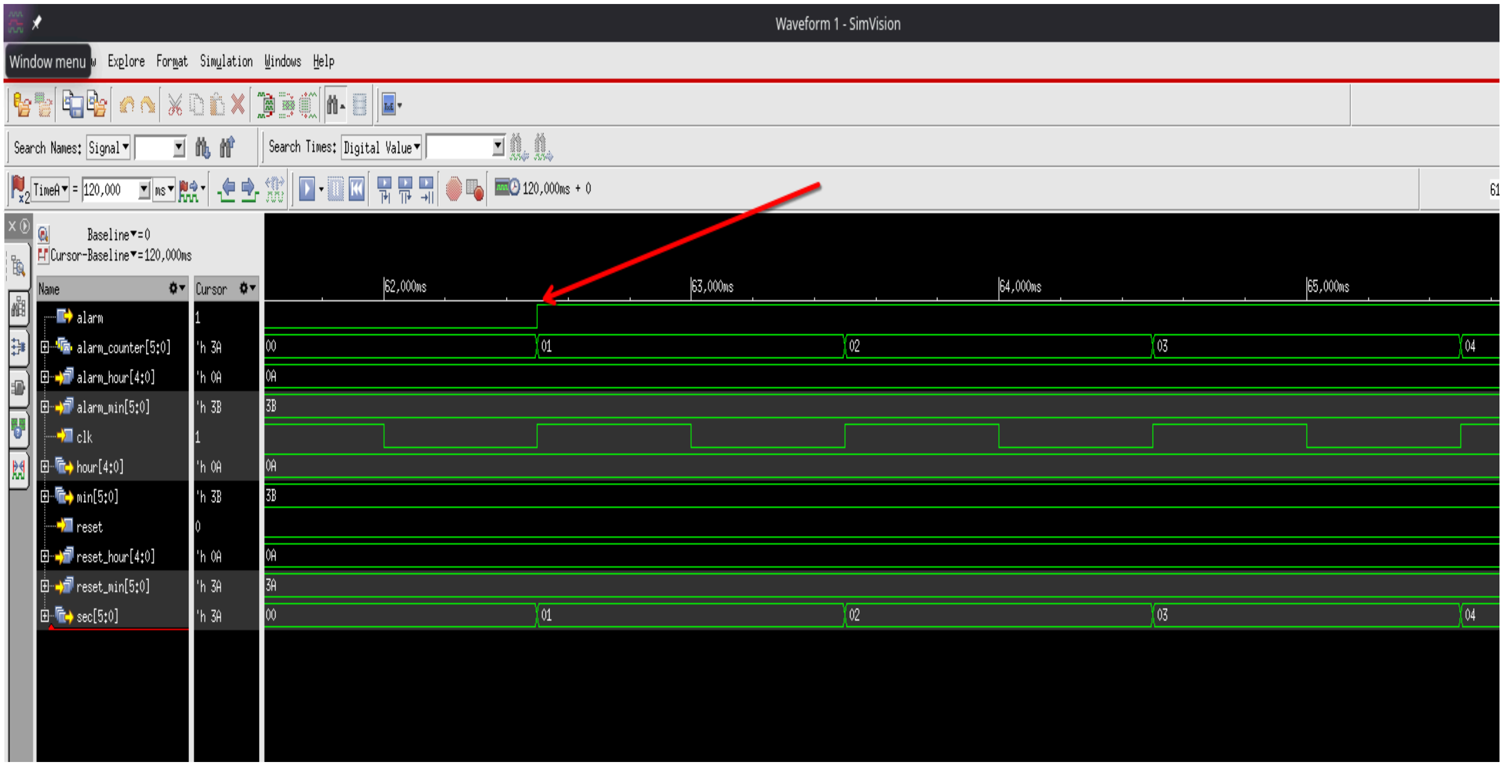

Fig01: Overall Timing Diagram of Alarm clock

RTL synthesis and reports of RTL synthesis

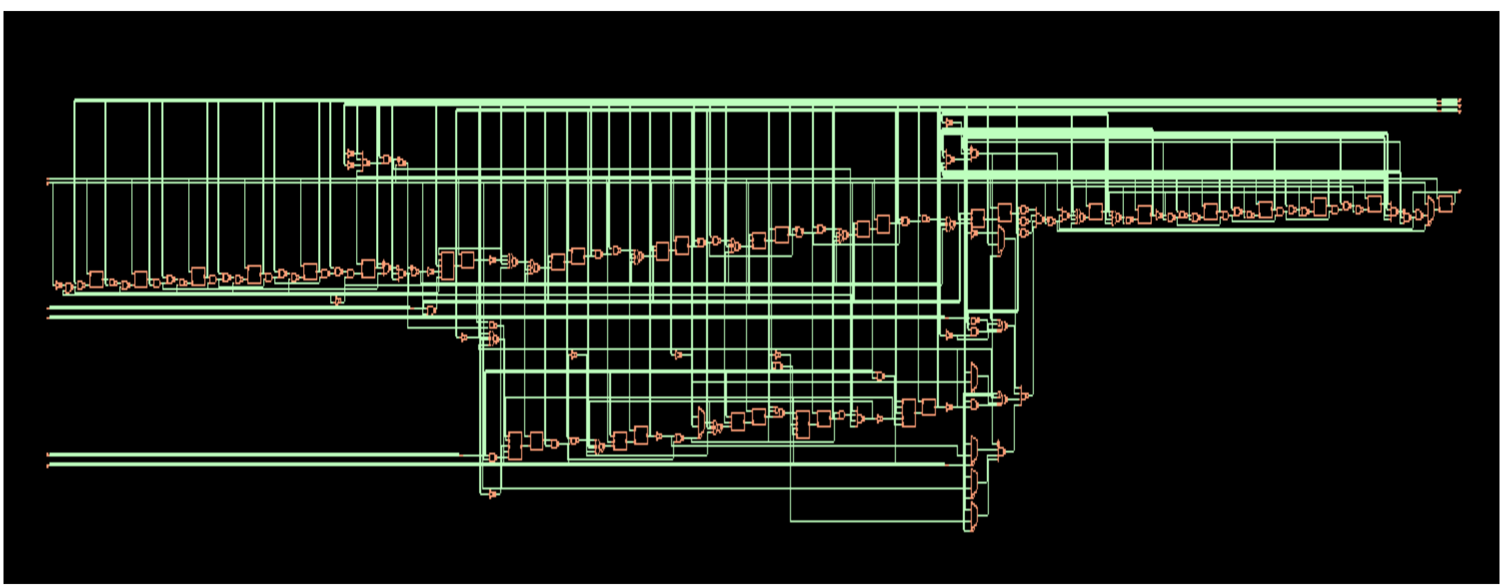

Fig02: Schematic View

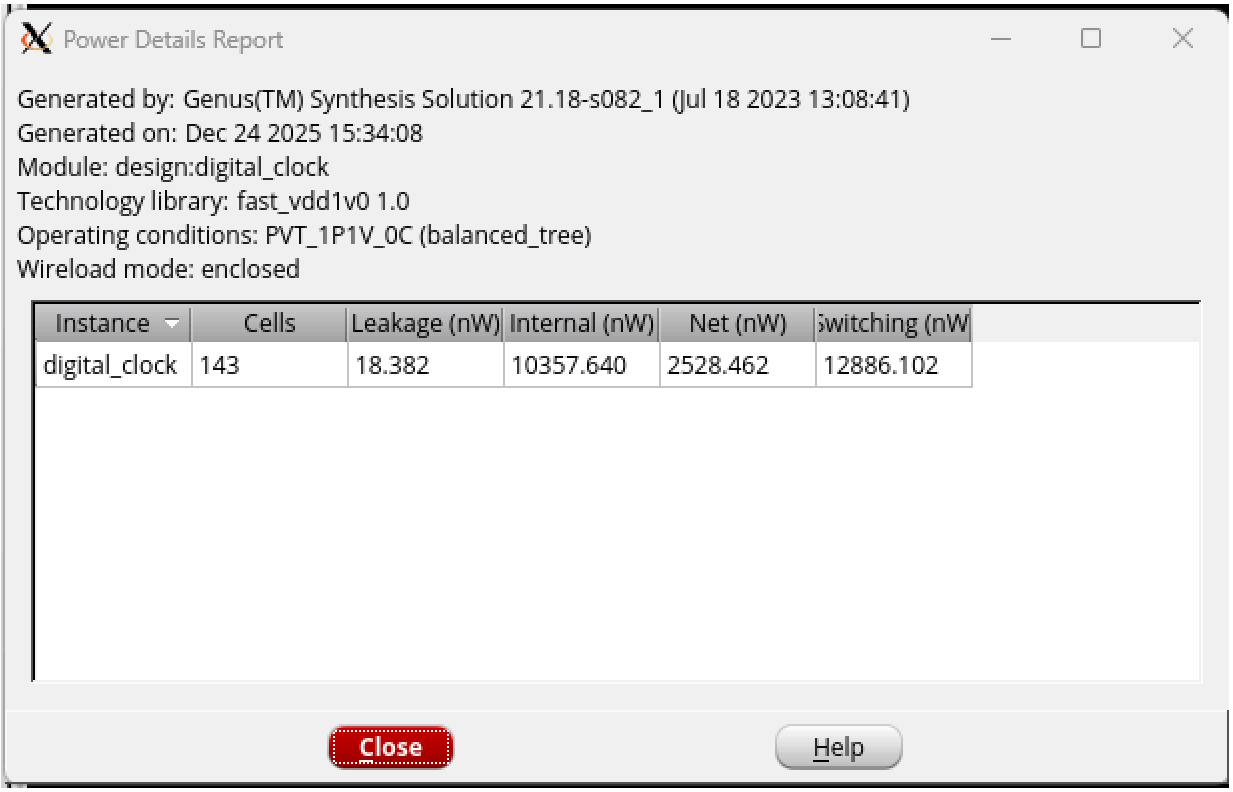

Fig03: Power Report for fast.lib

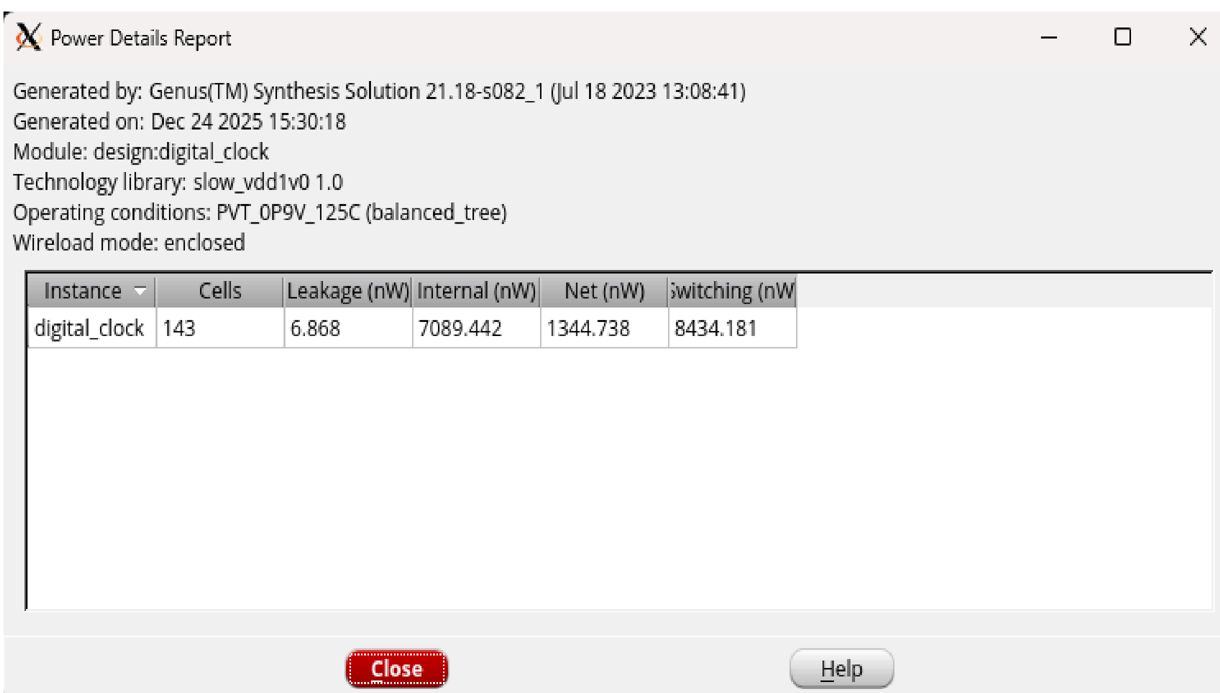

Fig04: Power Report for slow.lib

Physical Design

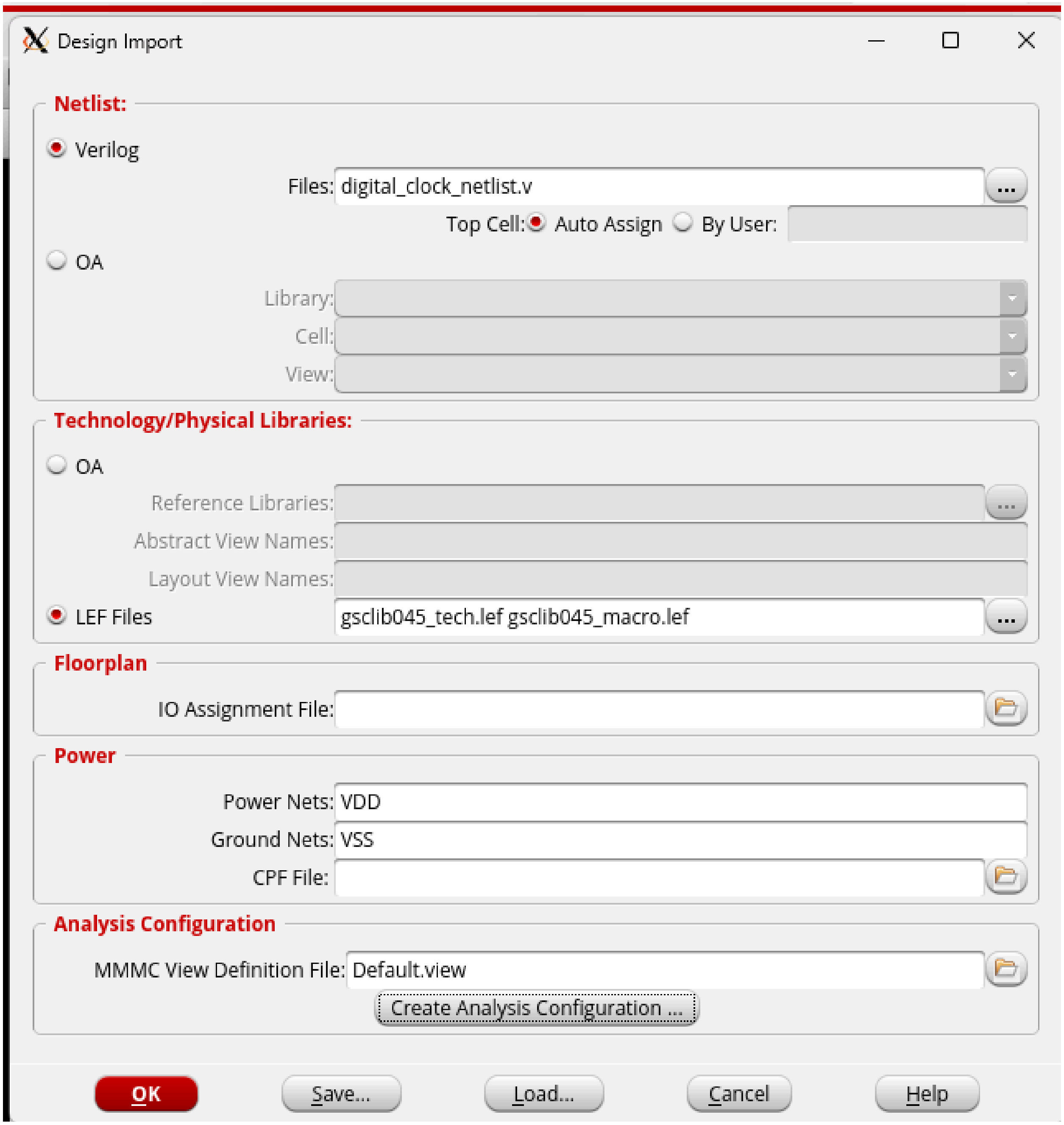

Fig05: Design Import Specifications

Fig05: MMMC Browser

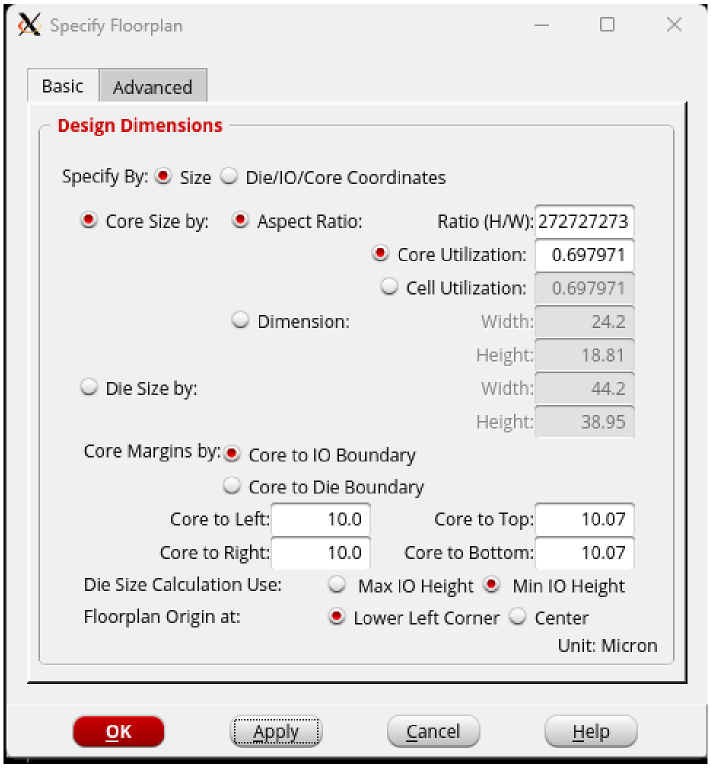

Fig06: Floorplan Settings

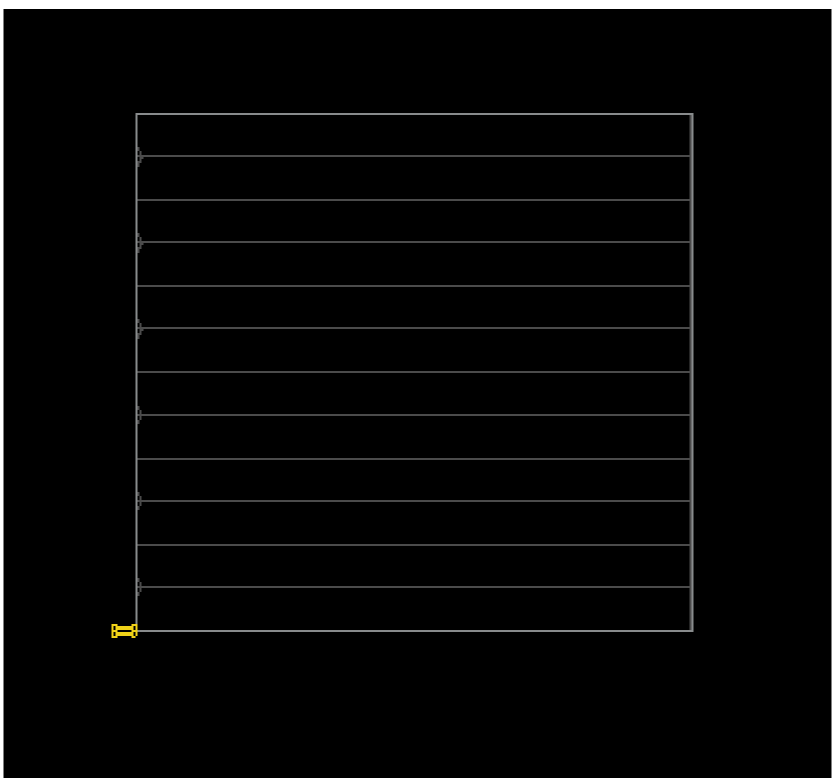

Fig07: Layout01

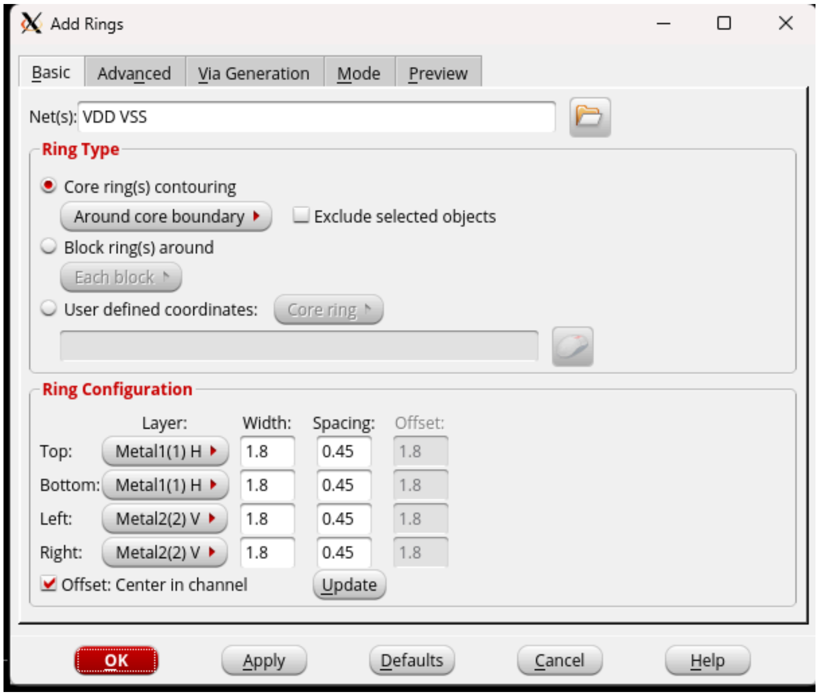

Fig08: Adding Ring

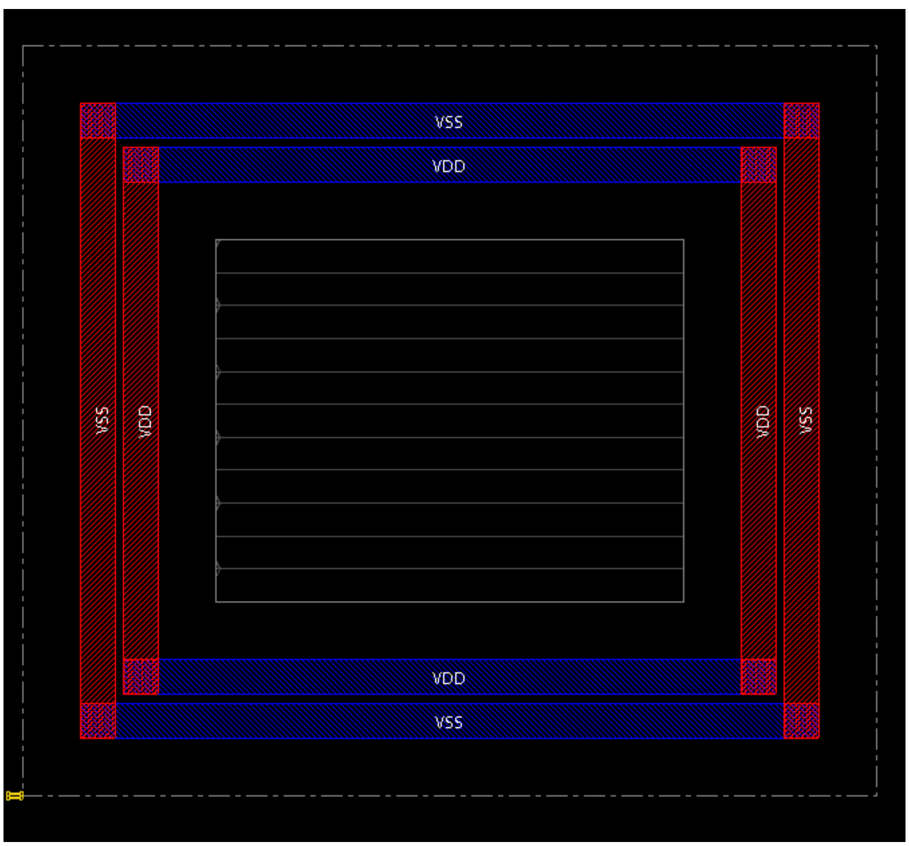

Fig09: Layout02

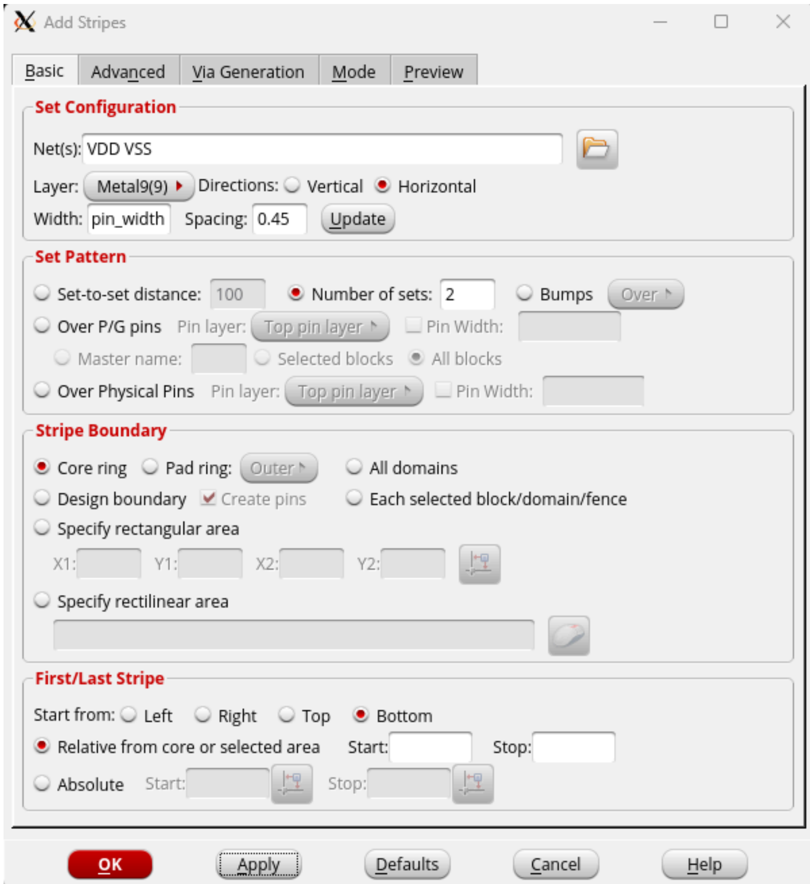

Fig10:Adding Stripes

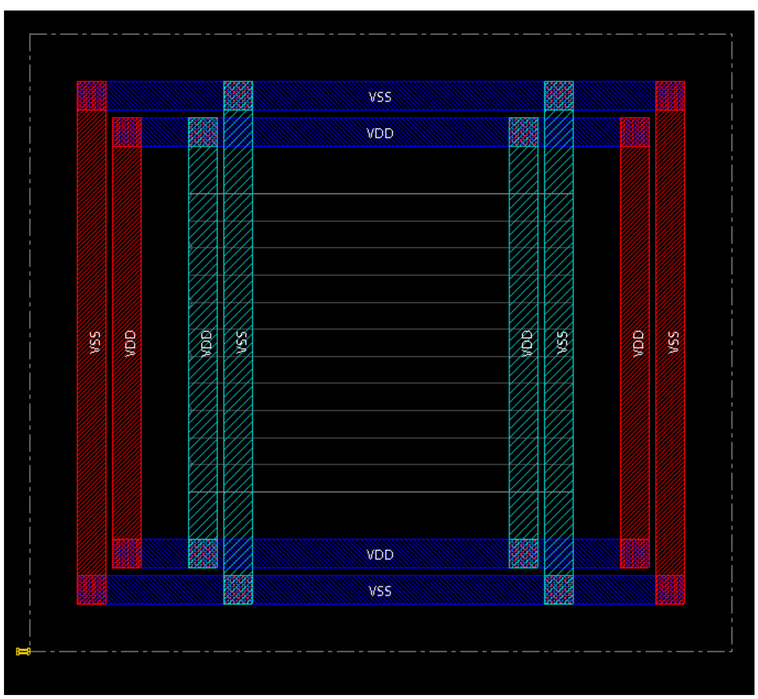

Fig11: Layout03

Fig12: Routing

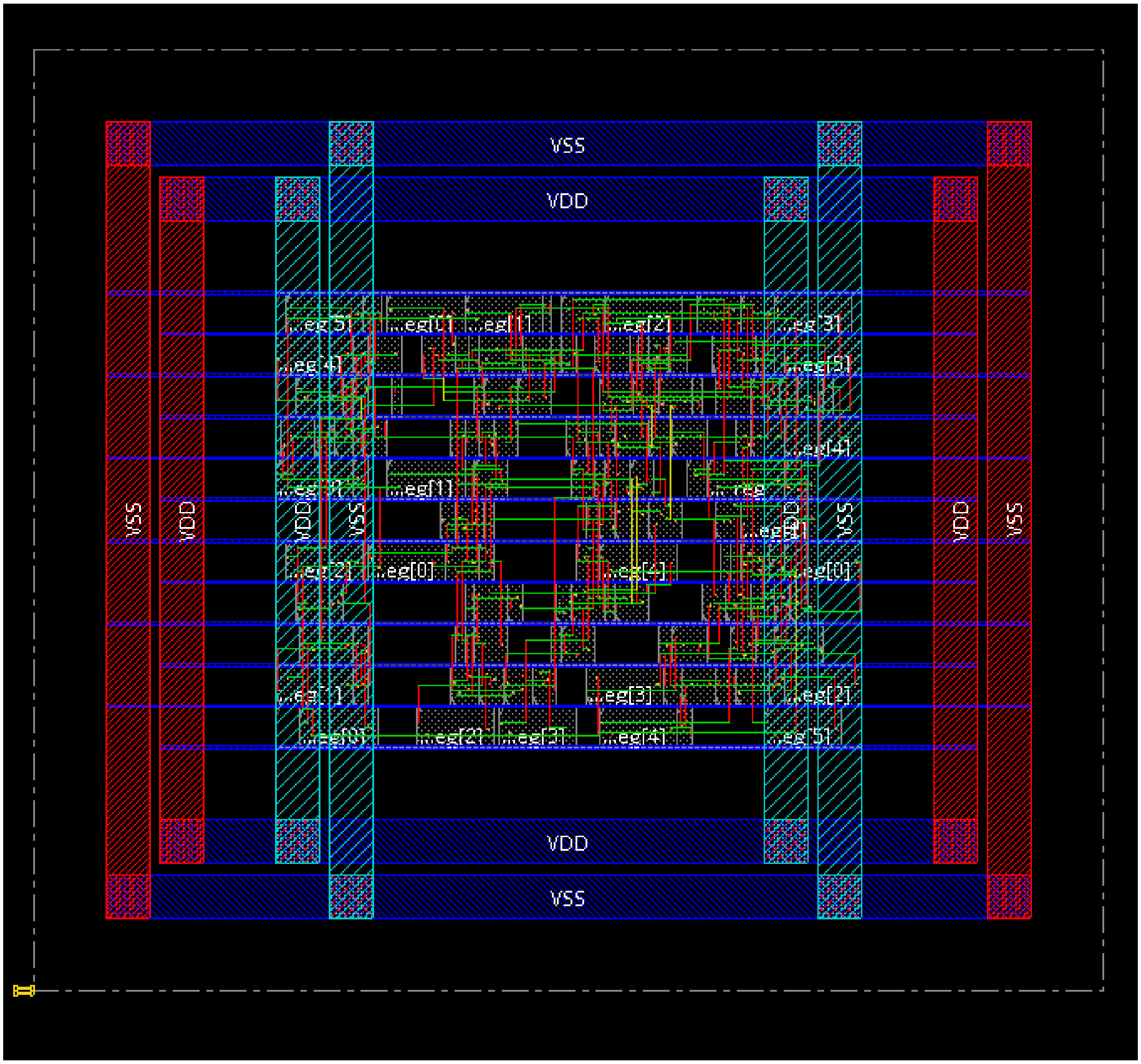

Fig13: Layout04

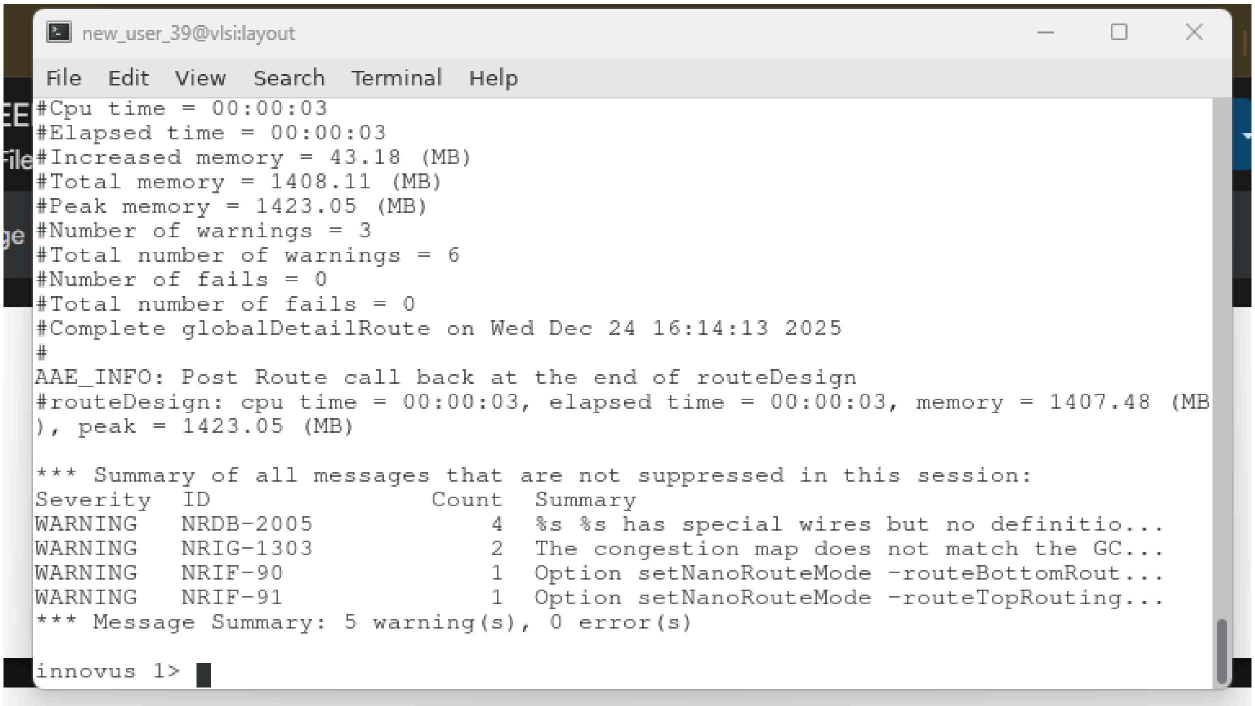

Fig14: No DRC Violation

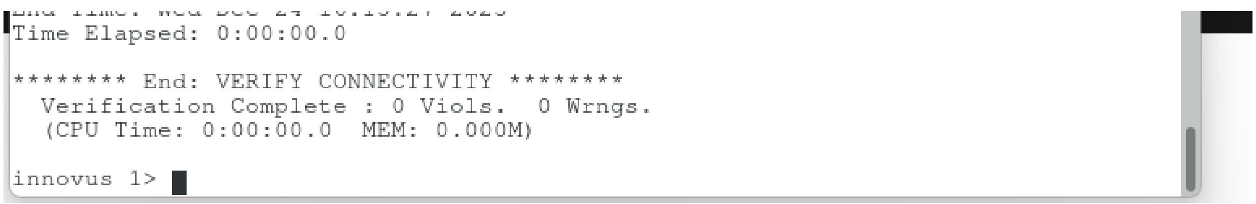

Fig15: Connectivity

Result and Observation

The timing diagram presented in this work demonstrates the successful implementation and validation of a digital clock system with alarm functionality. The simulation results captured from Cadence Waveform Viewer confirmed that all design requirements have been met and the system operates as intended.

The timing diagram displays the behavior of all critical signals over a simulation period of 0ns to 65,000ns.The following observations validate the correct operation of design:

The clock signal exhibits regular periodic pulses with consistent high and low phases providing the fundamental timing reference for the entire system. The clock operates at specified frequency and maintains stable transitions throughout the simulation period, which is essential for synchronous operation of all sequential elements. The reset signal remains at logic low(0) throughout the observed simulation window indicating normal operation. The second counter demonstrates proper incremental counting behavior advancing from 0 to 1, 2, 3 & 4 in sequential clock cycles This validates that the second counter is functioning correctly and incrementing the appropriate rate. The minute counter and the hour counter show the characteristics as the second counter so this validates the operation of time counting. The alarm time is properly configured with alarm_hour set to 'h 0A and alarm_minute set to 'h 3B. The alarm counter displays an incrementing sequence from zero to 6 synchronized with the seconds counter This indicates that the alarm comparison logic is actively monitoring the time and updating the alarm counter accordingly. In our timing diagram the cursor is set to the red arrow. Here we can see the hour mark is 'h 0A and the minute mark is 'h 3B' hence we can see the alarm variable is increased by 1. Here, we can say that our alarm clock design perfectly demonstrates the alarm functionality of the clock.

Discussion

During the design and implementation of the digital Alarm clock project several important technical and practical aspects of VLSI based digital system design were explored and understood in depth.
One of the main challenges faced was related to the alarm triggering mechanism. Without proper control the alarm could continuously retrigger every clock cycle while the hour and minute matched the alarm time. This issue was resolved by introducing an internal alarm counter which ensured the alarm was asserted only once per matching event and remained active for a fixed duration. This approach improved design robustness and demonstrated the importance of internal state variables in synchronous digital systems.

From the simulation and timing diagram analysis, it was observed that the design behaved exactly as intended when driven by a 1 Hz clock. The waveform validation helped confirm direct synchronization between seconds, minutes, hours and alarm signals. This reinforced the importance of test bench driven verification before moving to synthesis. During RTL synthesis, the project provided valuable insight into how high level Verilog descriptions are translated into hardware schematics. Analyzing the synthesized schematic and power reports helped in understanding resource utilization and timing behavior under different libraries. It also highlighted how design simplicity contributes to better power efficiency. The physical design phase including floor planning, adding rings, stripes and routing was particularly educational. It demonstrated how logical designs are transformed into real silicon layouts. Achieving a final layout with no DRC violations and correct connectivity emphasized the importance of proper power planning and routing strategies in VLSI design.

Overall, this project enhanced understanding of the complete VLSI design flow, from RTL coding and functional verification to synthesis and physical implementation. It also emphasized disciplined debugging, modular design thinking and the critical role of simulation and verification at every stage. The experience gained from this project will be highly beneficial for more complex digital and VLSI system designs in the future